SYSTÈME ABS

selmanim113@gmail.com
marouane0fareh@gmail.com

Mohamed SELAMANI
MAROUANE FAREH

TABLE OF CONTENT

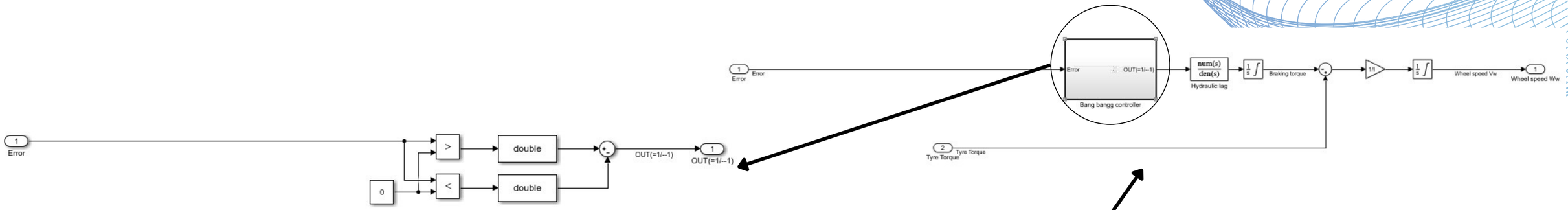
- Definition
  - Modeling of ABS in Matlab Simulink
    - Simulation in Matlab Simulink
      - Generate MATLAB's C code

DEFINITION

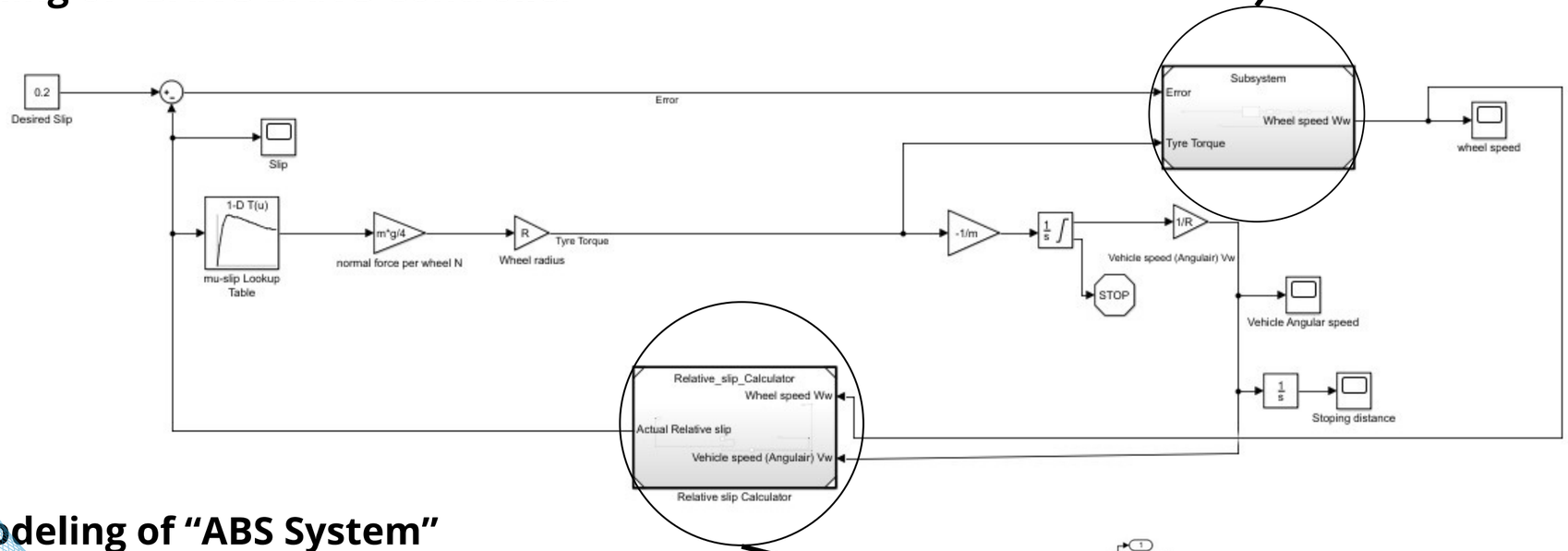
The Anti-lock Braking System (ABS)

The ABS system is an advanced vehicular safety technology that employs sensors to monitor wheel speeds and modulates the braking pressure on individual wheels through hydraulic control units (HCUs) in real-time. It is designed to detect and prevent wheel lockup by adjusting the braking force to optimize traction and steering control, especially in emergency or hard braking situations.

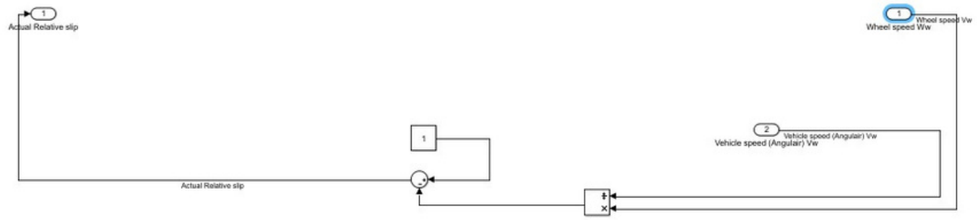
Modeling of ABS in Matlab Simulink



Modeling of “BANG BANG Controller”



Modeling of “ABS System”



Modeling of “Relative slip calculator”

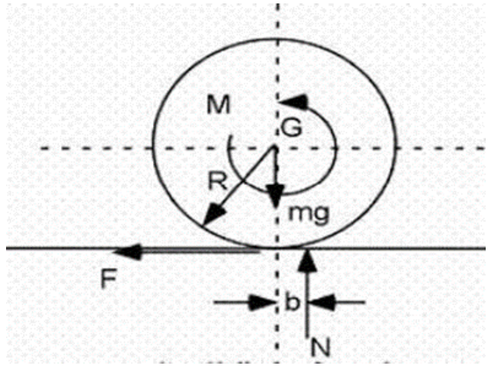
Determination of differential equations

$$-F_{fr} = m_v a_v$$

$$a_v = \frac{-4 * F_{fr}}{m_v}$$

$$F_{fr} = \mu * N = \mu * \frac{m_v * g}{4}$$

$$T_t = \mu . f_{fr} = \mu . R * \frac{M_v * g}{4}$$



$$T_t = \mu . f_{fr} = \mu . R * \frac{M_v * g}{4}$$

$$a' = \frac{1}{I} * (T_t - T_b)$$

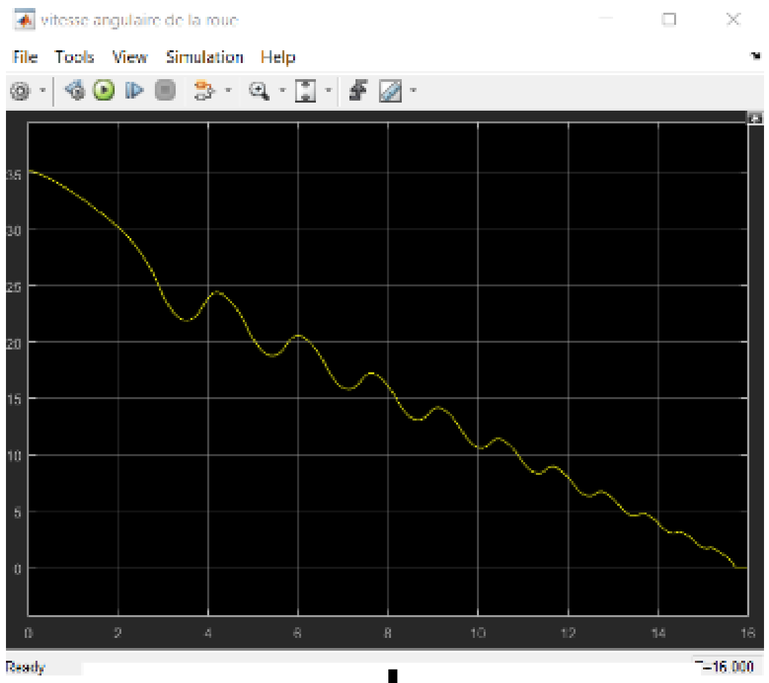
$$\omega_R = \int \frac{1}{I}(T_t - T_b)\,dt + \omega_0$$

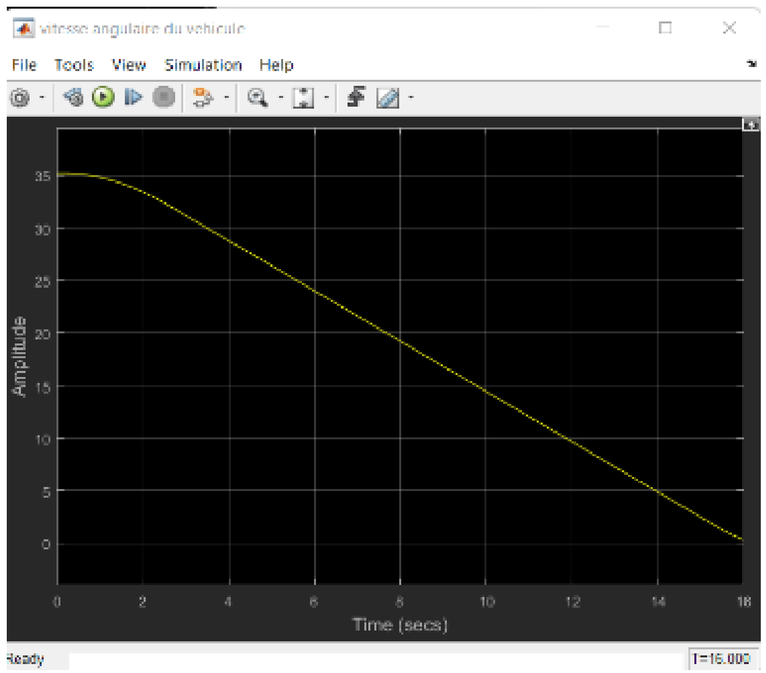
- μ : coefficient de frottement
- N : Effort normal
- Ffr : Force de frottement
- Av : vehicule acceleration
- Mv : vehicule mass
- a˙ : Angular accelaration
- Tt : Tyre torque
- Tb :Braking moment
- I: Inertie Moment
- R : Radius of the wheel
- mR : Vitesse angulair de la roue
- Vv : Vitesse lienaire du véhicule
- ωV : Vitesse angulaire du véhicule . ωV =Vv/R

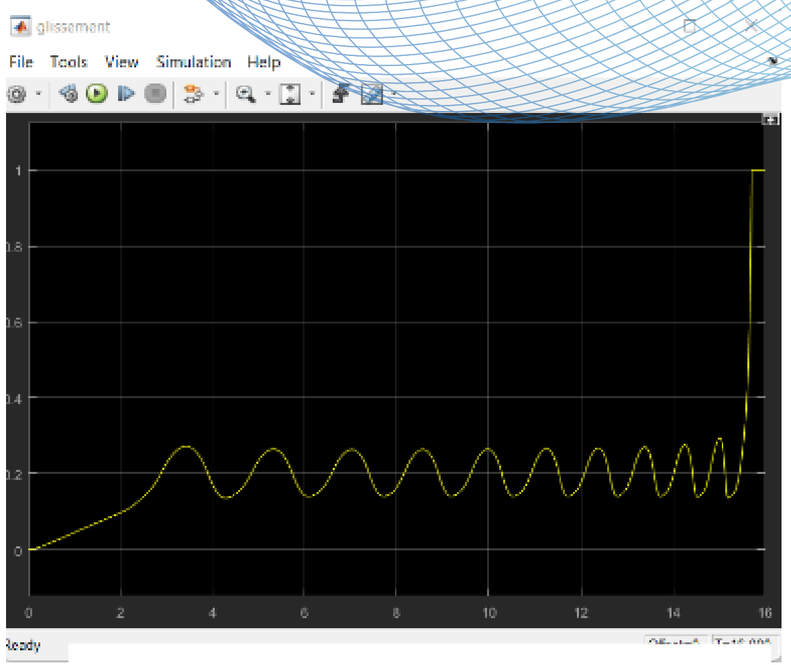
Relative slip:

$$S = \frac{\omega_v - R\omega}{\omega_v} \times 100 = 1 - \frac{\omega_R}{\omega_v}$$

Simulation in Matlab Simulink





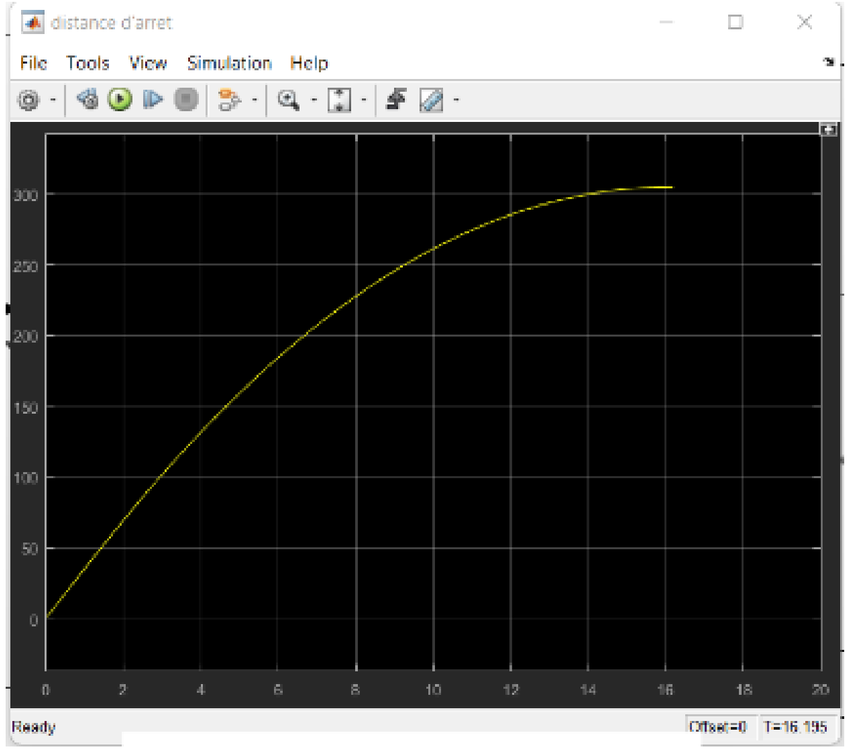


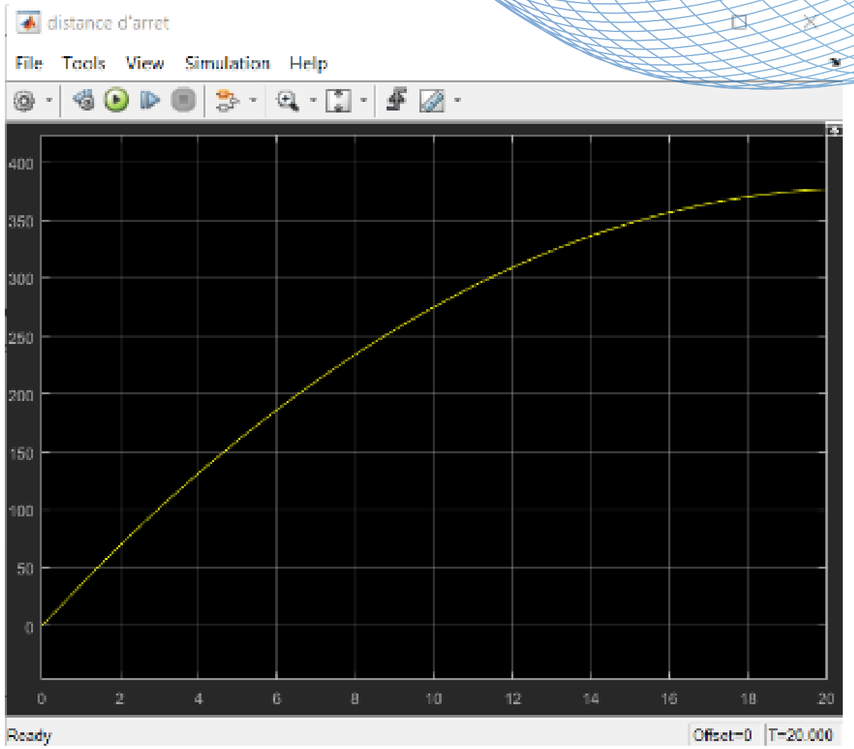
Wheel angular velocity

Vehicule angular velocity

Slip

Stoping distance’s Comparaison





Stoping distance with ABS

Stoping distance without ABS

• MATLAB's C code

```c
#include <stddef.h>
#include <stdio.h>
#include "Absl_restored_from_autosave.h" // Including the header file for the model
#include "rtwtypes.h"

void rt_OneStep(void); // Function prototype for a single step of the model

void rt_OneStep(void) {
    static boolean_T OverrunFlag = false; // Variable to detect overrun condition

    // Disable interrupts or other context switches here (if applicable)

    // Check for an overrun condition
    if (OverrunFlag) {
        rtmSetErrorStatus(rtM, "Overrun detected"); // Set error status in case of an overrun
        return; // Exit the function to prevent further execution
    }

    OverrunFlag = true; // Set the flag to indicate model step execution

    // Step the model
    Absl_restored_from_autosave_step(); // Execute a single step of the model

    // Get model outputs and handle any other necessary tasks here

    OverrunFlag = false; // Clear the flag after model step completion
}

int_T main(int_T argc, const char *argv[]) {
    // Unused arguments
    (void)(argc);
    (void)(argv);

    // Initialize the model
    Absl_restored_from_autosave_initialize(); // Initialize the model components

    // Simulate the model's behavior until an error occurs or stop is requested
    while ((rtmGetErrorStatus(rtM) == (NULL)) && !rtmGetStopRequested(rtM)) {
        rt_OneStep(); // Execute a single step of the model
    }
    // Disable rt_OneStep() or perform any cleanup tasks here if needed
    return 0; // Return success after the simulation completes
}
```